



Phase 1 — Déployer un modèle de classification

Du training à la production · Sentiment en français · TF-IDF → FastAPI → Docker



8h30 – 13h00



Groupes de 3-4

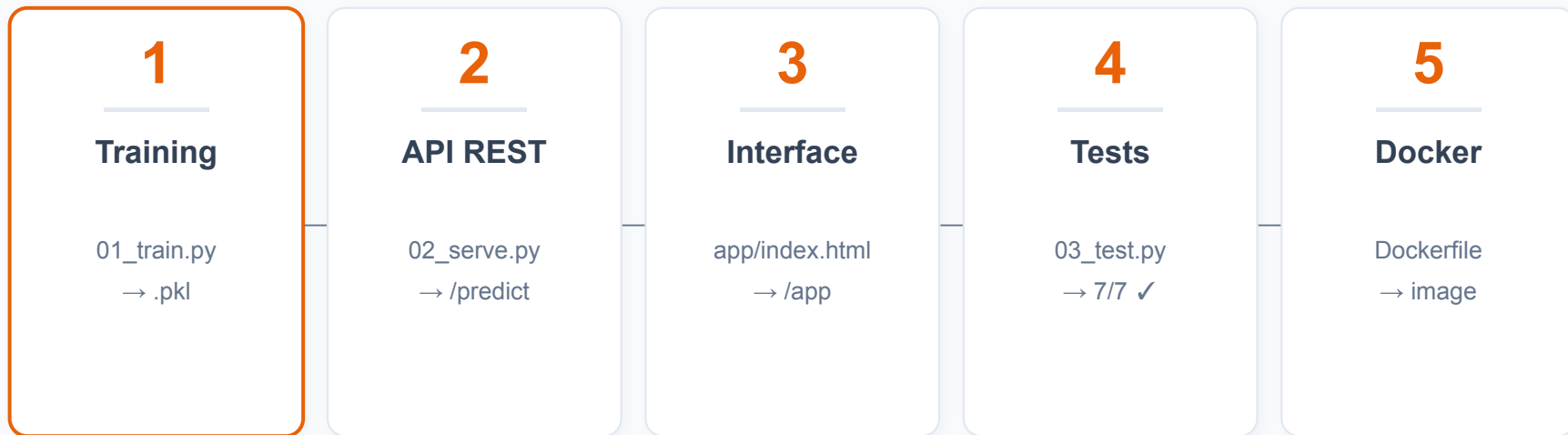


70% hands-on



7 volontaires

Le pattern de déploiement — 5 étapes, 1 seul schéma



Résultat attendu à 13h00 : un utilisateur non-technicien interroge le modèle sur <http://localhost:8000/app>

Même pattern applicable à CamemBERT, à un LLM, à YOLO — c'est la compétence de déploiement.

Setup — Avant de commencer

À faire avant la session ou en tout premier

1

Décompresser

```
unzip CITADEL_TP_Etudiant.zip && cd citadel_tp_etudiant
```

2

Environnement virtuel

```
python3 -m venv .venv  
source .venv/Scripts/activate # Windows Git Bash  
source .venv/bin/activate     # Linux / Mac
```

3

Dépendances

```
pip install -r requirements.txt  
# scikit-learn · fastapi · uvicorn · requests (~2 min)
```

4

Vérifier

```
python -c "import sklearn, fastapi; print('OK')"
```



Préparer les machines pour avoir un environnement prêt.

1

Training & Sérialisation

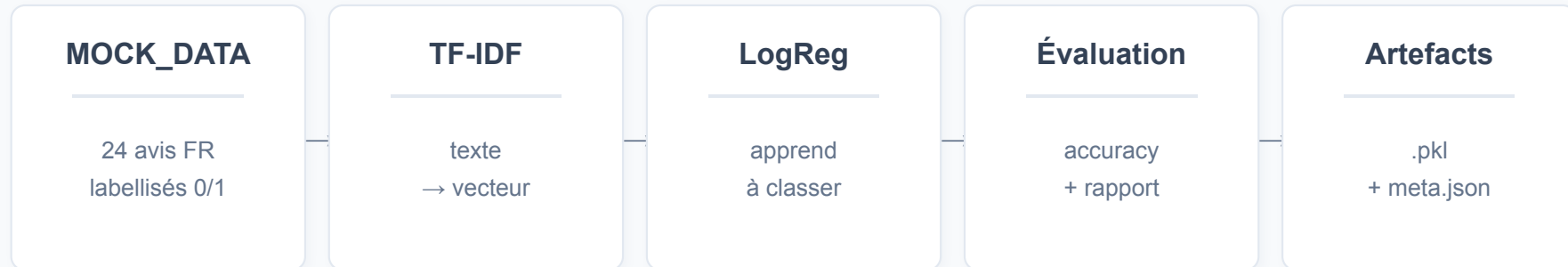
01_train.py · Compléter les 5 fonctions du pipeline ML

8h30 – 9h15 · 45 min ·

Volontaire 1

Que fait 01_train.py ?

Pipeline : données → vectorisation → entraînement → sauvegarde



Artefacts créés dans model/sentiment/

<code>vectorizer.pkl</code>	le TF-IDF entraîné — transforme n'importe quel texte en vecteur
<code>classifier.pkl</code>	les poids du classifieur — retourne 0 ou 1
<code>meta.json</code>	type, classes, accuracy — lu par l'API au démarrage

Hands-on V1 — 01_train.py

 Volontaire 1

#	Fonction	Ce qu'il faut écrire
1a	<code>preparer_donnees()</code>	<code>[t for t, _ in dataset] / [l for _, l in dataset]</code>
2	<code>creer_vectoriseur()</code>	<code>TfidfVectorizer(ngram_range=(1,2), max_features=5000)</code>
3a	<code>entraîner()</code>	<code>vectoriseur.fit_transform(textes_train)</code>
3b	<code>entraîner()</code>	<code>LogisticRegression(max_iter=1000).fit(X_train, labels)</code>
4a	<code>evaluer()</code>	<code>vectoriseur.transform(textes_test)</code> ← ⚠ pas <code>fit_transform</code>
4b-c	<code>evaluer()</code>	<code>predict()</code> + <code>classification_report()</code>
4d	<code>evaluer()</code>	<code>accuracy_score(labels_test, predictions)</code>
5a-c	<code>sauvegarder()</code>	<code>pickle.dump(vec)</code> / <code>pickle.dump(clf)</code> / <code>json.dumps(meta)</code>

✓ Résultat : Accuracy xx.xx% · 3 fichiers dans `model/sentiment/`

2

API REST

FastAPI

02_serve.py · Charger le modèle et exposer /predict

9h15 – 10h00 · 45 min ·

Volontaire 2

Pourquoi une API REST ?

Sans API

- ✗ Utiliser le modèle requiert Python installé
- ✗ Connaître le code source
- ✗ Réservé aux développeurs
- ✗ Impossible depuis un navigateur

Avec FastAPI

- ✓ Envoyer un JSON via HTTP
- ✓ N'importe quel langage / navigateur
- ✓ Modèle chargé une fois → rapide
- ✓ Séparation code ML / interface

Flux d'une requête :

```
POST /predict → { "texte": "Ce produit est excellent !" }  
               ↓ charge vectorizer.pkl + classifier.pkl (une fois)  
               ↓ transform → predict → predict_proba  
               ← { "label": "positif", "score": 0.94, "positif": true }
```


Hands-on V2 — 02_serve.py

#	Fonction	Ce qu'il faut écrire
1a	<code>charger_modele()</code>	<code>json.loads(meta_path.read_text()) → meta</code>
1b-c	<code>charger_modele()</code>	<code>pickle.load() → state['vectoriseur'] / state['classifieur']</code>
1d	<code>charger_modele()</code>	<code>state['meta'] = meta · state['classes'] = meta['classes']</code>
2a	<code>predire()</code>	<code>vec.transform([texte]) ← liste d'un seul texte</code>
2b	<code>predire()</code>	<code>clf.predict(X)[0] → label_id</code>
2c	<code>predire()</code>	<code>clf.predict_proba(X)[0][label_id] → score</code>
2d	<code>predire()</code>	<code>state['classes'][label_id] → 'positif' ou 'négatif'</code>
3	<code>lifespan()</code>	<code>charger_modele()</code> avant le <code>yield</code>
4a-c	<code>predire_sentiment()</code>	<code>valider · appeler predire() · retourner PredictionSortie</code>

3

Interface & Health

02_serve.py · /health endpoint + interface web /app

10h00 – 10h45 · 45 min ·

Volontaire 3

Hands-on V3 — /health + /app

 Volontaire 3

5 `etat()` Retourner {status, modele, accuracy, classes} depuis state



Vérifier /health

```
curl http://localhost:8000/health
# → {"status":"ok","modele":"tfidf+logreg","accuracy":0.833}
```



Ouvrir l'interface

```
http://localhost:8000/app
```



Lire le fetch() dans app/index.html

```
fetch('http://localhost:8000/predict', {
  method: "POST", body: JSON.stringify({ texte: input }),
  headers: { "Content-Type": "application/json" }
}) // ← zéro dépendance, n'importe quel navigateur
```

4

Docker

Dockerfile · Containeriser pour un déploiement reproductible

10h45 – 11h30 · 45 min ·

Volontaire 4

Hands-on V4 — Dockerfile

1

Lire Dockerfile.phase1 — commenter chaque ligne

```
FROM python:3.11-slim          # image de base légère (~150 Mo)
WORKDIR /app                   # répertoire de travail
COPY requirements.txt .         # copier en 1er → cache Docker préservé
RUN pip install -r requirements.txt
COPY phasel_sentiment/ ./phasel_sentiment/
CMD ["uvicorn", "02_serve:app", "--host", "0.0.0.0", "--port", "8000"]
```

2

Builder et lancer

```
docker build -f Dockerfile.phase1 -t citadel-sentiment .
docker run -p 8000:8000 \
  -v ./phasel_sentiment/model:/app/phasel_sentiment/model \
  citadel-sentiment
```



Si Docker absent : expliquer le concept, lire le Dockerfile, skip l'exécution.

5

Tests & Validation

03_test.py · Health check, prédictions, cas limites

11h30 – 12h15 · 45 min ·

Volontaires 5 & 6

Hands-on V5 & V6 — 03_test.py

 Volontaires 5 & 6

#	Fonction	Ce qu'il faut écrire
1a	<code>tester_health()</code>	<code>requests.get(f'{API_BASE}/health', timeout=5)</code>
1b	<code>tester_health()</code>	<code>if reponse.status_code != 200: return False</code>
1c-d	<code>tester_health()</code>	<code>reponse.json() / data.get('status') != 'ok'</code>
2a	<code>tester_prediction()</code>	<code>requests.post('/predict', json={'texte':texte}, timeout=10)</code>
2b-d	<code>tester_prediction()</code>	<code>reponse.json() → label_predit == label_attendu</code>
3	<code>tester_texte_vide()</code>	<code>requests.post('/predict', json={'texte':''}) → code 400</code>



Lancer les tests (terminal séparé)

```
python 03_test.py  # → 7/7 tests réussis
```

6

Débrief & Synthèse

Ce que vous avez construit · Questions de réflexion · Perspectives

12h15 – 13h00 · 45 min ·
Volontaire 7

Ce que vous avez construit

`01_train.py`

Un modèle entraîné

TF-IDF + LogReg, xx% accuracy, sauvegardé en .pkl

`02_serve.py`

Un service déployé

FastAPI expose /predict, /health, /app via HTTP

`app/index.html`

Une interface web

Testable sans Python, sans code, depuis le navigateur

`Dockerfile`

Un container

Image reproductible, déployable sur ANAM ou tout serveur Linux

`03_test.py`

Des tests automatisés

7 cas validés dont les cas limites

Le même pattern s'applique à CamemBERT, à Qwen3-8B, à YOLO. Vous venez d'apprendre à déployer n'importe

Questions de réflexion — Volontaire 7

Sérialisation

→ Pourquoi pickle ? Quelles sont ses limites de sécurité ?

transform vs fit_transform

→ Que se passerait-il si on utilisait fit_transform sur le test ?

Lifespan FastAPI

→ Que se passe-t-il si on charge le modèle dans /predict directement ?

Vers CamemBERT

→ Que faudrait-il changer dans 02_serve.py pour passer au vrai CamemBERT ?

Limites du modèle

→ Que retourne l'API pour un avis en mooré ? Est-ce acceptable ?

Phase 2

→ Quelle différence entre déployer LogReg et déployer Qwen3-8B ?

Phase 2 — Le même pattern, un LLM

Hackathon · RAG sur les arrêtés du MESRSI · Qwen3-8B via llama.cpp

Paramètre	Phase 1 (aujourd'hui)	Phase 2 (hackathon)
Modèle	TF-IDF + LogReg (~1 Mo)	Qwen3-8B-Q4 (5 Go)
Sérialisation	.pkl (pickle)	GGUF (llama.cpp)
Chargement	<code>pickle.load()</code>	llama-server (process séparé)
Endpoint	POST /predict	POST /ask
Latence	< 50 ms	5 – 30 s
Cas d'usage	Sentiment avis	RAG sur arrêtés MESRSI
Infrastructure	n'importe quel CPU	Cluster ANAM (480 cœurs)

 Le code de l'API (FastAPI, lifespan, endpoints) reste quasi identique. Seul le chargement du modèle change.

CITADEL

Frugal AI · Open Source · Made for Africa

Ouagadougou, Burkina Faso · 2026